These URLs are perfectly valid given the template's syntax, but some might cause problems for your application. These URLs have two or three segments, so the router happily assigns route values and matches the template when you might not want it to! These are the route values assigned:

- `/shoes/sneakers/test` has route values `category=shoes`, `name=sneakers,` and `id=test`.
- `/shoes/sneakers/123` has route values `category=shoes`, `name=sneakers`, and `id=123`.
- `/Account/ChangePassword` has route values `category=Account`, and `name=ChangePassword`.
- `/Cart/Checkout/Start` has route values `category=Cart`, `name=Checkout`, and `id=Start`.
- `/1/2/3` has route values `category=1`, `name=2`, and `id=3`.

Typically, the router passes route values to handlers through model binding, which you saw briefly in chapter 5 (and which chapter 7 discusses in detail). A minimal API endpoint defined as

```
app.MapGet("/fruit/{id}", (int id) => "Hello world!");
```

would obtain the `id` argument from the `id` route value. If the `id` route parameter ends up assigned a *noninteger* value from the URL, you'll get an exception when it's bound to the *integer* `id` parameter.